Top 11 Node.js Output-Based Interview Questions

Practice set covering JavaScript execution, Node.js, Express.js, REST APIs, MongoDB CRUD, JWT authentication and bcrypt.

Instructions: Predict the output or behaviour before running each snippet. Assume required packages, models, database connections and surrounding Express setup are available unless stated otherwise.

1. Event Loop Output

```
console.log("Start");

setTimeout(() => {
    console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
    console.log("Promise");
});

console.log("End");
```

Question: What is the output and why?

2. setTimeout vs Promise

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

setTimeout(() => console.log("D"), 0);

console.log("E");
```

Question: Predict the output.

3. Async/Await Output

```
async function getData() {
    console.log("1");
    await Promise.resolve();
    console.log("2");
}

console.log("3");
getData();
console.log("4");
```

Question: What will be printed and in which order?

4. Middleware Execution Order

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
    console.log("Middleware 1");
    next();
});

app.use((req, res, next) => {
    console.log("Middleware 2");
    next();
});

app.get("/", (req, res) => {
    console.log("Route");
    res.send("Hello");
});

app.listen(3000);
```

Question: What will be printed when a user visits /?

5. Middleware Without next()

```
app.use((req, res, next) => {
    console.log("First Middleware");
});

app.use((req, res, next) => {
    console.log("Second Middleware");
    next();
});

app.get("/", (req, res) => {
    res.send("Hello");
});
```

Question: Will the second middleware execute? Why?

6. REST API Status Code

```
app.post("/users", async (req, res) => {
    const user = await User.create(req.body);

    res.status(201).json({
        message: "User created",
        user
    });
});
```

Question: Which HTTP method is used? What does status code 201 mean? What response does the API return?

7. MongoDB CRUD Output

```
const user = await User.create({
    name: "Rahul",
    age: 22
});

console.log(user.name);

const updatedUser = await User.findByIdAndUpdate(
    user._id,
    { age: 25 },
    { new: true }
);

console.log(updatedUser.age);
```

Question: What will be the output, assuming the database operations succeed?

8. JWT Token Verification

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
    { id: "101", role: "admin" },
    "secret123"
);

const decoded = jwt.verify(token, "secret123");

console.log(decoded.id);
console.log(decoded.role);
```

Question: What will be printed? What happens if a different secret is used during verification?

9. JWT Authentication Middleware

```
const authMiddleware = (req, res, next) => {
    const token = req.headers.authorization?.split(" ")[1];

    if (!token) {
        return res.status(401).json({
            message: "Unauthorized"
        });
    }

    next();
};
```

Question: What happens if the token is missing? What happens if it is present? Does this middleware actually verify the JWT?

10. bcrypt Password Comparison

```
const bcrypt = require("bcrypt");

const password = "hello123";
const hashedPassword = await bcrypt.hash(password, 10);

const result = await bcrypt.compare(
    "hello123",
    hashedPassword
);

console.log(result);
```

Question: What will be the output? What happens if you compare hello456 instead?

11. Express Error Handling

```
app.get("/user", async (req, res, next) => {
    try {
        throw new Error("User not found");
    } catch (error) {
        next(error);
    }
});

app.use((err, req, res, next) => {
    res.status(500).json({
        message: err.message
    });
});
```

Question: Which middleware executes? What response does the client receive? Why do we use next(error)?